

Глава 1

Функциональное программирование

1.1. Немного предыстории

Функциональный подход, как и любые другие подходы к построению вычислений, отличается своей *полной* моделью представления алгоритмов как цепочки вычисления *функций*. Под функцией подразумевается обычная детерминированная функция одного или нескольких аргументов, с которой мы сталкиваемся в математике.

В основе функционального подхода лежит λ -исчисление, разработанное Алонзо Чёрчем в начале 30-х годов XX века. В начале того века большинство математиков испытывали головокружение от успехов формальной логики и были уверены, что некоторые парадоксы и противоречия математики и теории множеств, которые к этому времени успели накопиться, вполне себе разрешимы. Покончить с противоречиями решил Давид Гильберт (или его попросили). Основная его идея была в том, чтобы представить арифметику, а затем, и всю математику, как формальное исчисление, состоящее из аксиом и правил вывода, которые, если применять аккуратно, всегда давали бы непротиворечивые конструкции, и можно было бы проверить любое утверждение, написанное в некотором универсальном формальном синтаксисе без привязки к значениям слов или символов на истинность. И, таким образом, построить, вавилонскую башню, используя лишь кирпич и глину и силу строителей, зяждущих едиными устами. Но, как раз, формализация понятия алгоритма (сделанная одновременно Чёрчем и Тьюрингом) привела к доказательству того, что формальная логика не способна описать даже арифметику. Типичный пример – *проблема разрешимости*. Как и следовало, вавилонская башня оказалась разрушена, а программа Гильберта свёрнута. Гёдель со своей *теоремой о неполноте* плакал над её развалинами, ну а мы теперь наслаждаемся множеством языков (программирования), промыслительно замешанных для нас Богом ради непопущения нового вавилона.

Итак, с одной стороны, λ -исчисление (по крайней мере в том виде, в котором мы его собираемся использовать, т.е. *без типов*) очень простое, поскольку содержит в себе небольшое количество аксиом (базовых объектов) и правил вывода. Почти как кирпич и цемент (брение). А с другой стороны — оно достаточно мощное, чтобы описать все разнообразие алгоритмов. В его минималистичности и мощи – вся *красота*. Кстати, в этом плане, красота математических моделей схожа с красотой *оружия* – в нем нет ничего лишнего, и при этом в нем мощь. Но про конец вавилона надо помнить, и красотой, полнотой и прочими достоинствами не соблазняться.

С самим λ -исчислением предлагаю ознакомиться самостоятельно. В чистом виде оно пригодится тем, кто будет строить свой язык программирования. Нам важно выявить в нём несколько важных моментов, которые отличают функциональный подход от всего того, что было изучено ранее. А именно — от дальнейшего развития, которое получила машина Тьюринга (он также участвовал в разрушении вавилона) — императивного подхода к построению вычислений.

- F – Функция может создать и вернуть функцию, а также принимать функцию в качестве аргумента. Ещё говорят в этом случае, что функция является объектом *первого класса* (*first-class functions*).
- C – Функция может использовать значение аргументов функций, внутри которых была создана. Это свойство также называется *замыканием* (*closure*).

М – Нет состояний, переменных и прочих проявлений времени, либо они сильно локализованы, а их применение ограничено. Таким образом подчеркивается *неволатильность, неэфемерность* данных или *монотонность* вычислений (вычисление строит новый мир на обломках старого).

В дальнейшем, при изучении будем пользоваться буквами **F**, **C**, **M** для отсылки к этим особенностям.

1.2. Язык Scheme

Сам язык Scheme стал самостоятельно развиваться как один из (упрощённых) диалектов языка LISP, первоначально разработанный Гаем Стиллом и Джеральдом Сассманом в середине 70-х годов прошлого века, и использовался (даже до недавнего времени) в качестве образовательного пособия в MIT. Кстати, всем рекомендую обратиться к книгам-курсам SICP (Structure and Interpretation of Computer Programs) и HTDP (How to Design Programs). С тех пор, Scheme набирал популярность, и даже стал официальным языком расширений проекта GNU. Однако, "промышленным" языком программирования, в отличие от своего отца LISP, он не стал. В настоящее время за ним остались, в основном, исторические ниши академического языка, языка быстрого прототипирования и языка расширений (плагинов, скриптинга) некоторых программ.

Но Scheme — это язык-донор. Многие техники построения, идеи, я бы даже сказал, архитектуры программ на языке Scheme можно с успехом переносить в другие языки.

Более того, данный язык, в силу его природных свойств, (а именно, унаследованной от LISP *гомоиконичности* – когда код и данные имеют единый синтаксис) обладает непревзойденной системой *макросов*. Благодаря ей, возможно, определенная степень интереса к данному языку не угасает. Также, *макросы* являются исключительно удобным средством для формирования различных *предметно-ориентированных языков* (Domain-Specific Languages — DSL). Можно смело сказать, что DSL является полноценным, мощным *методом* программирования.

Ну, и как всякий язык с историей, он оброс *мультипарадигматичностью*. В нём, для облегчения труда программиста, естественно, сделано много отступлений от принципа **М** – есть и переменные, и циклы, и эфемерные структуры данных, и даже пристроены сбоку ООП и типы! Так что он вполне может потрафить любым вкусам. Даже таким, когда нужен именно такой халатик (например, ООП), но перламутровыми пуговицами (макросы). И всё же, идиоматичным сохраняется функциональный подход.

Основной бедой за всю долгую историю Scheme (и его прародителя LISP) является большое количество несовместимых между собой реализаций оных. В своих изысканиях мы будем пользоваться реализацией Guile Scheme. Большинство дистрибутивов Linux включают её естественным образом, но, обычно, старой версии – 1.8. В принципе, нам будет её достаточно. Более старые версии, например, 2.2.7 или 3.0 — содержат в себе множество улучшений в плане скорости и реализации дополнительных модулей. Взять свежачок можно здесь:

<https://www.gnu.org/software/guile/>

У guile есть режим REPL (Read, Evaluate, Print, Loop), который позволяет использовать guile в качестве "калькулятора" и тестирования конструкций.

1.3. Базовый конструктивный принцип

Итак, познакомимся с базовым конструктивным принципом. Основой синтаксиса языка является S-выражение. По сути — это список других S-выражений или термов, разделенных пробелами и заключенных в скобки. Пример:

(a b c)

Термами могут быть числа, строки, символы:

(a b (c d) e (f))

Таким образом, можно построить синтаксическое дерево любой сложности.

Также S-выражение является вызовом функции. Первый элемент – это сама функция, а оставшиеся элементы – её аргументы. Например:

```
(+ 1 2)
(modulo 10 3)
(* 10.0 (+ 8 (* (sqrt 2) 4) (- 2 1)))
```

В этом плане, в жертву универсальности принесена эргономика инфиксной математической записи, к которой мы привыкли за много лет.

Базовых типов данных в Scheme достаточно много:

- Башня чисел: целые, с плавающей точкой, рациональные (пишутся как дроби - $1/2$, $3/4$, $12093/98098$ и т.д), длинные целые, комплексные (записываются как $3.0+4.5i$).
- Строки (последовательности символов в двойных кавычках с экранированием): "hello world\n".
- Символы (*symbols*). Аналог именованных идентификаторов. Записываются как односложные слова с апострофом впереди: 'a, 'my-number или 'sum.
- Булевы значения: #t и #f.
- Отдельные (юникодные) символы (Chars), например #\tab, #\newline или #\x25CC.
- **Пары и списки.**
- Ключевые слова.
- Векторы.
- Структуры.
- и пр.

На парах и списках остановимся поподробнее. Пара (pair) является элементарным строительным блоком. Она объединяет два объекта. При помощи пар можно построить практически любые структуры, правда, не всегда эффективные. В частности, при помощи пар строится список, коим является также любое S-выражение. Согласно идее *гомоиконичности*, в Scheme код — это такие же данные, как и все остальное. Пара в Scheme обозначается как (a . b) (пара объектов, между которыми стоит точка, выделенная пробелами). Сконструировать пару можно при помощи процедуры cons:

```
(cons 3 4)
=>
(3 . 4)
```

Список в Scheme является односвязным списком пар, где в каждой паре в левой её части хранится элемент списка, а в правой – хвост списка. Например, список (1 2 3 4 5) выглядит вот так:

```
(1 .
  (2 .
    (3 .
      (4 .
        (5 .
          ())))))
```

Список всегда заканчивается пустым списком '(). Списки можно конструировать различными способами. При помощи процедуры list:

```
(list 1 2 3 4 5)
=>
(1 2 3 4 5)
```

При помощи формы *quote*:

```
(quote a b c 1 3)
=>
(a b c 1 3)
```

При помощи сокращения *'()* (синтаксический сахар для *quote*:

```
'(a b c 1 3)
=>
(a b c 1 3)
```

Правда, при этом будет построен список из символов (Symbols) 'a, 'b, 'c и чисел 1, 3. Что делать с этим дальше, мы рассмотрим чуть позже.

Значения внутри пары можно узнать при помощи функций *car* и *cdr*:

```
(car '(1 2 3))
=>
1
(cdr '(1 2 3))
=>
(2 3)
(car (cdr '(1 2 3)))
=>
2
(car (cons 1 2))
=> 1
(cdr (cons 1 2))
=> 2
```

Для удобства обращения к разным частям списка существуют сокращения:

```
(cadr x) => (car (cdr x)) ; второй элемент списка
(caddr x) => (car (cdr (cdr x))) ; третий элемент списка
(cdar x) => (cdr (car x)) ; хвост списка, если он является первым элементом
(cddar x) ...
...
```

Там где именованных функций не хватает, есть универсальный способ:

```
(list-ref '(1 2 3 4 5) 3)
=>
4
```

1.4. Функции

Функция создается при помощи формы *(lambda args body ...)*, где *args* – список аргументов функции, а *body* (и последующие за ним элементы) образуют тело функции. Функция возвращает результат, который дает последний элемент. Вот пример выражения, которое возвращает функцию возведения в квадрат:

```
(lambda (x) (* x x))
```

Как мы уже говорили, принцип **M** может, при желании, быть нарушен и такая запись:

```
(lambda (x) (* x x) (+ 2 x))
```

вернет функцию, которая будет прибавлять 2 к своему аргументу. Т.е. результат последнего выражения $(+ 2 x)$. Это означает, что все предыдущие выражения определения могут потенциально оставлять *побочный эффект* (изменять общее состояние). В нашем случае, результат $(* x x)$ будет просто отброшен. В чистой функциональной парадигме побочных эффектов быть не должно, а следовательно, нет надобности в *последовательности* выражений. Но в Scheme эта возможность есть, и слава Богу.

Функцию можно применить к аргументу:

```
((lambda (x) (* x x)) 20)
=>
400
```

Но можно присвоить значение функции (саму функцию, а не результат) некоторому глобальному символу с помощью формы `(define name val)`.

```
(define square (lambda (x) (* x x)))
```

Теперь можно вычислить квадрат при помощи функции *square*:

```
(square 3)
=>
9
```

Заметим, что при помощи *define* можно связывать с символом не только функции, но и любые другие объекты Scheme. Например, можно определить "переменную" *pi*:

```
(define pi 3.14159265)
```

и в дальнейшем использовать её в вычислениях.

1.5. Условия

Условное выполнение можно обеспечить формой `(if cond pos neg)`. Здесь *cond* – это выражение с условием, *pos* – выражение, которое будет вычислено если условие истинно и *neg* – если условие ложно.

```
(lambda (x) (if (< x 0) (- x) x))
```

Для проверки нескольких условий подряд существует форма `(cond (check expr ...) ... (else expr ...))`. Например:

```
(lambda (x y)
  (cond
    ((= y 0) 0)
    ((= x 0) 0)
    ((> x y) (/ x y))
    (else (/ y x))))
```

1.6. Рекурсия

Со средствами рекурсии в Scheme тоже всё хорошо:

```
(define fib (lambda (n)
  (if (< n 2) 1
      (+
        (fib (- n 1))
        (fib (- n 2))))))
```

```
(fib 3)
=>
3
(fib 5)
=>
8
```

Здесь есть, правда, повод для смущения, поскольку `fib` ещё не определён, а мы его уже используем. Но смущаться не надо, поскольку `scheme` использует позднее связывание, т.е. ищет определения на момент исполнения.

Приведённый выше способ вычисления n -го числа Фибоначчи не совсем оптимален, поскольку не использует хвостовую рекурсию. Мы можем переписать его в хвостовом варианте:

```
(define fib (lambda (n)
  (define fib' (lambda (m a b)
    (if (< m n)
        (fib' (+ m 1) (+ a b) a)
        a)))
  (fib' 1 1 1)))
```

Как видите, у нас появилось внутреннее определение `fib'`, которое является локальным для внутренних первой `lambda`. Мы определили функцию внутри функции и тут же воспользовались ей.

Так, к слову, чтобы часто не писать

```
(define f (lambda (x y) ...))
```

можно воспользоваться упрощённой формой:

```
(define (f x y) ...)
```

суть останется прежней, но будет чуть поменьше скобок.

1.7. Замыкания

Чтобы подробнее познакомиться с замыканиями `C`, рассмотрим следующий пример. Допустим, вы решили обобщить операцию умножения на какие-то часто используемые числа. Например, 1000, 1000000, 0.001 и т.п. В этом случае можно создать функцию – фабрику умножителей, чтобы потом использовать их в работе:

```
(define (multiplier n)
  (lambda (x) (* n x)))
(define mega (multiplier 1000000))
(define kilo (multiplier 1000))
(define milli (multiplier 0.001))
```

```
(mega 3)
=>
3000000
(kilo 5)
=>
5000
(milli 100)
=>
0.1
```

Таким образом, функция, возвращаемая функцией *multiplier* запоминает значение *n*, которое было передано ей в качестве аргумента. Можно сказать, что созданные функции *mega*, *kilo*, *milli* являются замыканиями своего контекста. Более того, при желании, можно добиться того, что контекст можно модифицировать (нарушая принцип **M**) так, что все модификации будут доступны другим участникам "захвата" контекста. Обычно приводят пример с банковским счётом:

```
(define (account balance)
  (list
    (lambda (debit)
      (set! balance (- balance debit))
      balance)
    (lambda (credit)
      (set! balance (+ balance credit))
      balance)
    (lambda () balance)))
```

Тут нам сразу бросается в глаза процедура *set!* (с восклицательным знаком на конце) — это модификатор значения символа. Восклицание означает нарушение принципа монотонности **M**. Это вынужденная мера. Без неё невозможно продемонстрировать "общение" между замыканиями через общую переменную. Главное, чтобы мы аккуратно следили за появлением побочных эффектов и ограничивали их расползание, если захотим по-максимуму сохранить функциональность и детерминированность.

Итак, как работает модификация окружения в замыкании. Создадим счёт:

```
(define poog (account 0))
```

Мы помним, что *poog* — это список из трёх функций, который нам вернула функция *account*. Первая — дебет, вторая — кредит, третья — запрос баланса. Сначала запросим у бедолаги баланс:

```
((caddr poog)) ; вызов третьей функции из списка poog
=>
0
((cadr poog) 100) ; положим бедному на баланс 100 рублей
=>
100
((caddr poog)) ; еще раз спросим баланс
=>
100
((car poog) 30) ; снимем немного
=>
70
```

В общем, у нас получился полноценный объект в смысле ООП, который сохраняет состояние и меняет его под действием сообщений.

Ещё один пример — это функции-генераторы. Для примера возьмём всё те же числа Фибоначчи:

```
(define (gen-fib)
  (let ((a 1) (b 1))
    (lambda ()
      (set! a (+ a b))
      (set! b a)
      b)))
```

Тут использована форма `(let ((var definition) ...) body ...)` которая позволяет вычислить *body* ... при условии, что *var = definition* (можно задать несколько определений). Можно сказать, что мы задали значения "переменных" *a* и *b*, которые доступны для изменения при помощи *set!*. Попробуем погенерировать:

```

(define seq1 (gen-fib))
(seq1)
=>
1
(seq1)
=>
2
(seq1)
=>
3
(seq1)
=>
5

```

Понятно, что про детерминированность тут можно забыть.

```

(define seq2 (gen-fib))
(seq2)
=>
1
(seq1)
=>
8

```

Как видим, все замыкания помнят свои состояния.

1.8. Обработка списков

Стоит напомнить, что всякое проявление времени в ФП нежелательно, поэтому всякий цикл, который бы мы использовали в императивном подходе преобразовывается в рекурсию. Благо, что этому способствует и рекурсивная природа списка. Рассмотрим пример обобщенных функций для работы со списками.

Функция `(map proc lst ...)` применяет к каждому элементу списка *lst* функцию *proc* и возвращает список из результатов вычисления. Например:

```

(map (lambda (x) (* x 2)) '(1 2 3 4 5))
=>
(2 4 6 8 10)

```

Если списков несколько, то *proc* вызывается с несколькими аргументами:

```

(map (lambda (a b) (* a b)) '(1 2 3 4 5) '(5 4 3 2 1))
=>
(5 8 9 8 5)

```

Функцию *map* можно написать самим:

```

(define (my-map proc lst)
  (if (null? lst) '()
      (cons (proc (car lst)) (my-map proc (cdr lst)))))

```

Функция `(fold proc init lst ...)`¹ позволяет произвести ”свёртку” списка *lst* с аккумулятором, последовательно вычисляя *proc* между аккумулятором и элементом списка. В начале значение аккумулятора равно *init*. Пример:

```

(fold + 0 '(1 2 3 4 5))
=>
15

```


Для свёртки важно, в какой последовательности производятся вычисления, поэтому есть ещё вариант *fold-right*. Второй вариант применяет *proc* начиная с конца списка. Для демонстрации разницы, приведем 2 примера:

```
(fold cons '() '(1 2 3 4 5)) ; аккумулярование начинается с начала списка
=>
(5 4 3 2 1) ; вычисляется как (cons 5 (cons 4 (cons 3 (cons 2 (cons 1 '())))))
(fold-right cons '() '(1 2 3 4 5))
=>
(1 2 3 4 5) ; вычисляется как (cons 1 (cons 2 (cons 3 (cons 4 (cons 5 '())))))
```

В принципе, все однопроходные циклы для обработки списков и даже более сложных структур можно обобщить при помощи *fold* и *fold-right*.

Задание

1. Напишите хвостовой вариант функции *map* не прибегая к прямым модификациям. Удастся ли вам сохранить порядок следования результата?
2. Напишите свой вариант *fold* и *fold-right*.

1.9. Быстрая сортировка

Ещё один пример, который показывает удобство работы со списками — это быстрая сортировка. Поскольку мы не собираемся без особых причин нарушать **М**, то выберем сортировку *слиянием*. Принцип такой сортировки — разделить список на две части, отсортировать по отдельности и слить обратно.

Разделить список на два можно рекурсивно, отдавая в результирующие списки по очереди (то в первый, то во второй его элементы).

```
(define (qsort-split lst)
  (let loop ; это аналог функции, которая может быть вызвана внутри let
    ; а это её параметры
    ((i lst) ; начальный список
     (l '()) ; левый результирующий список
     (r '())) ; правый результирующий список
    (cond
      ((null? i) ; если список пуст
       (list l r)) ; возвращаем список из двух результирующих списков
      (else ; иначе
       ; вызываем loop с новыми параметрами
       ; всегда добавляем голову i в l, но каждый раз меняем l и r местами
       (loop (cdr i) ; новый i -- это теперь его хвост
              r ; l -- это r
              (cons (car i) l)))))) ; r -- это приставленная голова i к l
```

Слияние происходит аналогично, но с учётом порядка:

```
(define (qsort-merge left right)
  (let loop ; наша рекурсивная функция
    ((l left) (r right)) ; её начальные параметры
    (cond
      ((null? l) r) ; если левый список пуст -- возвращаем правый
      ((null? r) l) ; аналогично, если наоборот
```

¹ в ядре guile её почему-то нет, но можно подключить модуль (use-modules (srfi srfi-1)) и она появится!

```

; если голова l меньше головы r,
; то отрываем её и вставляем в голову результирующего списка
; а хвост -- строим рекурсивно из хвоста l и r
((< (car l) (car r)) (cons (car l) (loop (cdr l) r)))
(else (cons (car r) (loop l (cdr r))))))

```

Финальная функция сортировки, соответственно, будет выглядеть так:

```

(define (qsort lst)
  (cond
    ((null? lst) lst) ; пустой список возвращаем как есть
    ((null? (cdr lst) lst) ; список из одного элемента не сортируем
    (else
     (apply qsort-merge (map qsort (qsort-split lst))))))

```

Здесь функция *apply* применяет *qsort-merge* к списку аргументов, который формируется при помощи *qsort-split* с последующей сортировкой *qsort*. Это более простой аналог следующего выражения:

```

(define (qsort lst)
  (cond
    ((null? lst) lst)
    ((null? (cdr lst)) lst)
    (else
     (let* ((split (qsort-split lst))
            (l (qsort (car split)))
            (r (qsort (cadr split))))
       (qsort-merge l r))))

```

1.10. Макросы

Достаточно уникальной вещью в Scheme являются *макросы*. Точнее даже, это *трансформаторы синтаксиса*. В применении выглядят они как обычные функции, но принцип их работы другой – они принимают на вход код, как он есть, в виде S-выражений, и на выходе выдают новый код, который уже затем интерпретируется.

Чтобы показать разницу между функцией и макросом, попробуем написать свой аналог *if* в виде функции.

```

; Внимание! это антипример!
(define (my-if check positive negative)
  (if check positive negative))

; пробуем применить
(my-if (< 3 2) (display "Total destroy!\n") (display "Peace\n"))
=>
Total destroy!
Peace
; Обычный if работает как надо
(if (< 3 2) (display "Total destroy\n") (display "Peace\n"))
=>
Peace

```

Язык Scheme использует *строгую* модель вычислений, к которой мы привыкли в императивных языках. В этой модели все аргументы, если они переданы в виде выражений в функцию, вычисляются перед вычислением самой функции. Существует противоположная модель вычислений – *ленивая*, при которой выражение вычисляется как только действительно понадобилось его значение, например, при выводе. И передается из функции в функцию именно в виде выражения. Такая модель реализована в языке Haskell.

В нашем гипотетическом случае *my-if* — это функция. И интерпретатор не передаст ей управления, пока не будут вычислены *все* её аргументы. Можно сэмулировать *ленивость* через λ -конструкции:

```
(define (my-lambda-if check positive negative)
  (if (check) (positive) (negative)))

(my-lambda-if (lambda () (< 3 2))
              (lambda () (display "Total destroy\n")))
              (lambda () (display "Peace\n")))
=>
Peace
```

Безусловно, в этом случае мы достигаем нужного поведения, путём задержки вычисления через оборачивание в λ . Но, очевидно, это предполагает много писанины и, в конце концов, запутывает.

Давайте теперь посмотрим как устроены трансформаторы синтаксиса, или, по-простому, макросы в Scheme. Для создания макроса используется конструкция *define-syntax*.

```
(define-syntax my-if
  (syntax-rules ()
    ((_ check pos neg) (if check pos neg))))

(my-if (< 3 2) (display "Total destroy!\n") (display "Peace\n"))
=>
Peace
```

В этом случае мы ничего не трансформировали, но просто переписали *my-if* на *if*. Форма *(define-syntax symbol rules)* определяет, что символ *symbol* является макросом и для трансформации его аргументов будут применены правила. Список правил трансформации может задаваться при помощи формы:

```
(syntax-rules literals (pattern transformation) ...)
```

Здесь *literals* — это список "строковых констант", которые не являются переменными в шаблонах *pattern*. Можно задать трансформацию *transformation* для различных шаблонов (*pattern*) вызова макроса.

```
(define-syntax generate
  (syntax-rules (for in if)
    ((_ expr for x in a)
     (map (lambda (x) expr) a))
    ((_ expr for x in a if b)
     (map (lambda (x) expr) (filter (lambda (x) b) a)))))

(generate (* x x) for x in '(1 2 3 4 5))
=>
(1 4 9 16 25)
(generate (* v v) for v in '(1 2 3 4 5) if (odd? v))
=>
(1 9 25)
```

Здесь мы задали макрос *generate* (аналог Питоновских list-comprehensions), который имеет два шаблона: *(generate expr for x in a)* и *(generate expr for x in a if b)*. Причем, "переменными" шаблона, которые привязываются к выражениям являются именно *expr*, *a* и *b*. а *for*, *in* и *if* — это строковые константы, которые обязаны встретиться при сопоставлении реальных выражений с шаблоном, но не участвуют в связывании. В первом случае, переменная *expr* имеет значением выражение *(* x x)*, переменная *x* — выражение *x*, а переменная *a* — *'(1 2 3 4 5)*. Во втором случае, переменная *expr* — *(* v v)*, переменная *x* — *v*, переменная *a* — *'(1 2 3 4 5)*, а переменная *b* — *(odd? v)*. Далее, они трансформируются в соответствующие выражения с *map* и *filter*.

В результате, в интерпретацию поступают два выражения:

```
; (generate (* x x) for x in '(1 2 3 4 5))
(map (lambda (x) (* x x)) '(1 2 3 4 5))
; (generate (* v v) for v in '(1 2 3 4 5) if (odd? v))
(map (lambda (v) (* v v)) (filter (lambda (v) (odd? v)) '(1 2 3 4 5)))
```

На самом деле, возможности трансформации синтаксических форм безграничны. Есть возможность обрабатывать повторяющиеся элементы при помощи символа `...`. Например, рассмотрим один из способов создания DSL, скажем, для написания музыки:

```
(define-syntax pattern
  (syntax-rules (/)
    ((_ name (num / denom) ((note ...) ... ))
     (define name (make-pattern num denom
                               (list (quote (note ...)) ... ))))))

(pattern gamma (8 / 16)
  ((a-3 b-3 c-4 d-4 e-4 f-4 g#4 a-4)
   (c-4 d-4 e-4 f-4 g#4 a-4 b-4 c-5)))
```

Данная конструкция трансформируется в

```
(define gamma (make-pattern 8 16
  (list '(a-3 b-3 c-4 d-4 e-4 f-4 #{g#4}# a-4)
        '(c-4 d-4 e-4 f-4 #{g#4}# a-4 b-4 c-5))))
```

Таким образом, `...` могут организовать циклы сопоставления с шаблоном и трансформации с сохранением вложенности: цикл по списку `note` `...` находится внутри цикла по списку списков `(note ...)` `...`.

Задания

1. Используя *define-syntax*, реализуйте свою версию *cond*.
2. Используя *define-syntax*, реализуйте форму `(for <x> in <lst> do <expr> ...)` которая бы последовательно вычисляла список выражений `<expr> ...` для каждого значения переменной `<x>` из списка `<lst>`. Подсказка: список выражений вычисляется при помощи формы

```
(begin <expr> ...)
```

1.11. Ассоциативные списки

В Scheme есть родная (функциональная, в смысле монотонная) структура данных — ассоциативный массив *alist*. Внутри он устроен просто как список пар ключ-значение (поэтому его правильно называть *ассоциативный список*), работа с ним проста и незатейлива. Обычно ассоциативный массив устроен так:

```
((key . value) (key2 . value2) (key3 . value3))
```

В качестве ключей могут выступать практически любые объекты, для которых может быть определена операция эквивалентности². Добавление элемента в ассоциативный список эквивалентно приставлению головы к списку при помощи *cons*:

²Эквивалентность в высокоуровневых языках может быть разная. В Scheme, например, есть минимум 4 операции сравнения: `=`, `eq?`, `eqv?` и `equal?`. Первый вариант `=` используется исключительно для проверки на равенство чисел. Попытки сравнить что-либо другое приведут к ошибке. Вариант `eq?` используется как быстрое сравнение объектов по указателю (аналог оператора `==` в Си). Т.е. `(eq? a b)` вернет истину только если `a` и `b` — имена одного и того же объекта. Вариант `eqv?` проверяет на равенство тип и значение объектов (для чисел). Числа разных типов не равны по `eqv?` если они имеют равные значения. Например `(eqv? 1 1.0) => #f`. Вариант `equal?` проверяет на равенство тип и содержимое объектов. Подходит для сравнения строк и даже списков.

```
(define a '((Masha . 10) (Misha . 20) (Vitya . 30)))
(cons '(Kolya . 40) a)
=>
((Kolya . 40) (Masha . 10) (Misha . 20) (Vitya . 30))
```

Можно догадаться, что поиск в ассоциативном списке осуществляется линейно, начиная с головы. Для поиска элемента используются функции *assq*, *assv* и *assoc*. Различаются они тем, что используют для сравнения, соответственно: *eq?*, *eqv?* и *equal?*. Если элемент с искомым ключом найден, то возвращается соответствующая пара, если нет — возвращается *#f*. В нашем случае в качестве ключей используются символы, поэтому нам вполне достаточно *assq*.

```
(define a '((Masha . 10) (Misha . 20) (Vitya . 30)))
(assq 'Masha a)
=>
(Masha . 10)
```

```
(assq 'Misha a)
=>
(Misha . 20)
```

```
(assq 'Albert a)
=>
#f
```

Для конструирования ассоциативного списка можно использовать функцию (*acons* *key* *value* *alist*), которая эквивалентна (*cons* (*cons* *key* *value*) *alist*).

Ничего страшного, если пара с одинаковым ключом встретится несколько раз. На то она и функциональная структура данных, что ничего не теряется (пока об этом не забудут). Семейство функций *assq* будут возвращать всегда первый по списку (а значит, последний добавленный) элемент.

Итерироваться по ассоциативному списку можно так же как и по обычному списку. Все рекурсивные алгоритмы, применимые к обработке списков будут работать и с ассоциативным списком.

Часто возникает необходимость взять только значение по ключу из списка. Для этого в модуле (*srfi* *srfi-1*) определено семейство функций *assq-ref*, *assv-ref* и *assoc-ref*. Будьте внимательны, порядок их аргументов отличается от аналогичных *assq*:

```
(use-modules (srfi srfi-1))
(define a '((Masha . 10) (Misha . 20) (Vitya . 30)))
(assq-ref a 'Masha)
=>
10
```

```
(assq-ref a 'Albert)
=>
#f
```

В этом модуле также определены функции для немонотонных изменений ассоциативного списка (например, *assq-set!*, *assq-remove!* и пр. но мы их здесь рассматривать не будем).

Конечно, такие списки не могут использоваться там, где требуется хранение большого количества пар ключ-значение. Линейное время поиска убивает производительность. Если нужна скорость, лучше использовать классические хеш-таблицы с временем доступа $O(1)$. Но, естественно, хеш-таблицы уже не являются чисто функциональными структурами данных, поскольку их изменение подразумевает *замену* содержимого. Неизменяемые хеш-таблицы (*vhash*) определены в модуле (*ice-9* *vlist*). В данном модуле, ради сохранения монотонности, изменение таблицы подразумевает создание её заново. Есть также и *эфемерные* хеш-таблицы, которые, кроме как немонотонно, изменить не получится.

Сохранение функциональности, при увеличении скорости, возможно только при использовании деревьев поиска.

Глава 2

Scheme и Предметно-Оrientированные ЯЗЫКИ

2.1. Немного о DSL

Пока все наши декларации о силе гомеоморфности, присущей Scheme, выглядели абстрактно. Теперь попробуем привести пример построения DSL. Как уже было сказано, построение DSL является *методом* решения сложных задач, по аналогии с методом разбиения задачи на подзадачи или принципом *разделяй и властвуй*. Использовать DSL, как метод, можно также и там, где самого DSL не требуется.

Обычно, построение какого-либо предметно-ориентированного языка связывают с *эргономикой* описания объектов или явлений какой-то предметной области, при этом чётко определяя границы этой области. Правильно провести границу — это искусство. Расширение границ приводит к усложнению языка, а сужение — к потере полноты и неэффективности. Т.е. хороший предметно-ориентированный язык *сбалансирован* относительно простоты и полноты описания. Часто, помимо объектов самой предметной области, для проведения хорошей границы приходится разрабатывать удобные абстракции — и это тоже часть метода.

Если вам часто приходится решать множество задач, которые можно было бы объединить в некоторое параметризованное семейство, то описание параметров, если оно нетривиально, может вполне быть кандидатом в DSL. Если у вас есть уже сформировавшаяся предметная область (не обязательно связанная с ИТ), которую вдруг потребовалось автоматизировать, то DSL, включающий в себя модель этой предметной области, также является естественным решением.

2.2. Пример языка нечёткой логики

В качестве примера DSL рассмотрим язык описания нечёткой логики (fuzzy logic). Нечёткая логика сама по себе является методом решения задачи упрощения описания поведения сложных многопараметрических функций, в качестве параметров и значений которых выступают обычно скалярные величины. Упрощение описания происходит за счет уменьшения точности. Т.е. поведение функции описывается *качественно* без привлечения математических формул. Нечёткая логика часто используется в бытовой технике (кондиционеры, стиральные машины) и в автомобилестроении (управление трансмиссией) для программирования контроллеров и управления оборудованием.

Что такое *нечёткая логика*? Вообще говоря, нечёткая логика отличается от бинарной тем, что вместо булевых значений Истина и Ложь оперирует степенью истинности (или вероятностью), которая характеризуется числом $0 \dots 1$. Соответственно, все логические функции модифицируются под работу с этим диапазоном, но таким образом, чтобы на границах диапазона (0 и 1) быть совместимыми с булевыми. Например, $\text{AND}(a, b)$ преобразуется в умножение $(a * b)$, $\text{OR}(A, B) \rightarrow \max A, B$, $\text{NOT}(A) \rightarrow (1 - A)$.

Упрощение строится за счет *факторизации* — разбиения процесса вычисления на 3 этапа: уменьшение размерности, вычисления в меньшей размерности и восстановление размерности (по аналогии с декомпозицией матриц). Этап уменьшения размерности называется *фаззификацией* пере-

менных — преобразование всего диапазона значений каждой переменной в *нечёткие множества*. Множество значений разбивается на "качественную" шкалу значений (например, температура может быть "Высокой", "Средней", "Низкой"). Каждое такое множество именуется n и сопровождается функцией принадлежности $\mu_n(x) \in (0 \dots 1)$. Функция принадлежности показывает, в какой степени значение x можно считать принадлежащей n . Например, в какой степени значение 45° является "Высоким". Эта функция специально выбирается не бинарной, а непрерывной. Она должна принимать значение 0, если x нельзя считать принадлежащей множеству n , 1 — если x с высокой степенью достоверности принадлежит n , и все значения в промежутке, если принадлежность можно считать не полной. Обычно для простоты используют кусочно-линейные функции $\mu_n(x)$ в виде "треугольника" или "трапеции" и распределяют нечёткие множества так, чтобы функции принадлежности перекрывались своими наклонными участками. Таким образом, образуются нечёткие границы множеств в которых x может принадлежать нескольким множествам с дробной степенью.

Промежуточный этап вычислений представляет из себя множество логических правил — импликаций в виде *ЕСЛИ условие ТО следствие*. Условие формируется в виде логической формулы, термами которой являются значения принадлежности той или иной переменной тому или иному значению нечёткой шкалы, а в качестве связок используются логические (модифицированные под непрерывный диапазон значений) функции (AND, OR, NOT). Следствие — это факт принадлежности результирующей переменной какому-то нечёткому множеству. Например, если мы управляем скоростью вращения вентилятора кондиционера *speed* в зависимости от температуры *temp* и влажности *humid*, то некоторые из правил могут выглядеть так:

```
IF (temp is High) AND (humid is High) THEN (speed is VeryHigh)
IF (temp is High) AND ((humid is Medium) OR (humid is Low)) THEN (speed is High)
...
```

Понятно, что результатом формулы в условии будет некоторое число от 0 до 1, смысл которого — достоверность, с которой результирующая переменная может принадлежать заданному значению шкалы. После применения всех правил, на выходе мы получим вектор принадлежностей V_n .

Финальный этап — это дефаззификация. Преобразование вектора принадлежностей в скалярное значение. Сделать это несколько труднее. Сначала строится весовая функция $w(y)$, которая обычно определяется как

$$w(y) = \max_n V_n \mu_n(y)$$

Затем, в качестве финального значения Y рассчитывается "центр масс" весовой функции:

$$Y = \frac{\int y w(y) dy}{\int w(y) dy}$$

Есть также вариант брать не центр масс, а точку наивысшего значения $w(y)$

$$Y = \underset{y}{\operatorname{argmax}} w(y)$$

Попробуем приспособить Scheme для описания и вычисления нечёткой логики. Для начала, хотелось бы придумать красивую эргономичную форму описания. Будем исходить из того, что дерево синтаксиса у нас уже есть — это S-выражения. Осталось только сочинить нужные формы. В нашей модели описания нечёткой логики есть входные и выходные переменные с описанием нечётких множеств с названиями и функциями принадлежности, а также правила вывода. Проэтому проще всего свести их в одну форму. Например:

```
(fuzzy outvar (var1 ...) rule1 ...)
```

где *var1 ...* — описание переменных, *outvar* — название выходной (результирующей) переменной, *rule1 ...* — список правил. Переменные можно описать в виде:

```
(varname (label1 func1 param ...)
         (label2 func2 param ...)
         ...)
```

Здесь *varname* — имя переменной, *label1* — имя нечеткого множества, *func1* — вид функции принадлежности, *param ...* — её параметры.

Поскольку импликации касаются, в основном, вывода значений выходной переменной *outvar*, правила можно облечь в форму:

```
(label1 if expression1)
(label2 if expression2)
...
```

Логические выражения *expression1* можно создавать в виде связок: *(and term1 ...)* — логическое умножение, *(or term1 ...)* — логическое сложение, *(not term1)* — логическое отрицание. Причем в качестве *term1* могут выступать как термы принадлежности значений переменной нечеткому множеству (*var1 is label1*), так и вложенные выражения.

Вот общий пример описания, который хотелось бы получить:

```
(fuzzy speed
  ((temp (low trap -200 -200 -20 5)
        (med trap -20 5 5 15)
        (high trap 5 15 60 60))
   (humid (low trap 0 0 10 20)
         (med trap 10 20 40 50)
         (high trap 40 50 100 100))
   (pres (low trap 0 0 700 750)
        (med trap 700 750 760 780)
        (high trap 760 780 1000 1000))
   (speed (low trap 0 0 100 200)
         (med trap 100 200 300 400)
         (high trap 300 400 500 500)))
  (high if (and (temp is high) (or (humid is high) (pres is low))))
  (med if (and (temp is med) (or (humid is high) (not (pres is med)))))
  (low if (or (temp is low) (humid is low) (pres is high))))
```

Желательно, чтобы вся эта конструкция ”компилировалась” в функцию, в которую можно было бы передать аргументы *temp*, *humid*, *pres* и получить значение *speed*:

```
(define speed-func (fuzzy ... ))
(speed-func '((temp . 30) (humid . 50) (pres . 750)))
=>
230.4
```

Прежде, чем приступить непосредственно к синтаксису, нужно подготовить модель вычислений.

2.2.1. Функции принадлежности

Для начала рассмотрим примеры реализации функций принадлежности $\mu(x)$. Наиболее употребительные – трапециевидная *trap* и треугольная *tri*. Это просто будут генераторы функций, которые принимают скалярное значение показателя и возвращать степень принадлежности:

```
(define (trap a b c d)
  (lambda (x)
    (cond ((< x a) 0)
          ((< x b) (/ (- x a) (- b a)))
          ((<= x c) 1)
          ((< x d) (/ (- d x) (- d c)))
          (else 0))))

(define (tri a b c)
```



```
(lambda (x)
  (cond ((<= x a) 0)
        ((< x b) (/ (- x a) (- b a)))
        ((< x c) (/ (- c x) (- c b)))
        (else 0))))
```

2.2.2. Фаззификация

Описание нечётких множеств можно представить в виде ассоциативного списка, где в качестве ключа используется имя множества, а в качестве значения – функция принадлежности. Например¹:

```
(define temp `((low . ,(trap -10 -10 0 10))
               (med . ,(tri 0 10 30))
               (high . ,(trap 10 30 40 50))
               (extreme . ,(trap 40 50 70 70))))
```

Соответственно, когда нам нужно проверить значение параметра на принадлежность нужному множеству, достаточно выбрать аргумент по имени из ассоциативного списка аргументов и применить к нему функцию из ассоциативного списка - описания переменной:

```
; допустим, аргументы в переменной args:
(define args '((temp . 30) (humid . 50) (pres . 750)))

; Проверим, насколько temp принадлежит low:
((assq-ref temp 'low) (assq-ref args 'temp))
=>
0
```

2.2.3. Правила вывода

С правилами вывода, с одной стороны всё просто, а с другой стороны – всё сложно. Просто, потому что (and x ...) нужно преобразовать в (* x ...). Логическое сложение (or x ...) – во взятие максимума (max x ...). А отрицание (not x) – в разность (- 1 x). Сложно – потому, что это нужно сделать рекурсивно. Но, при помощи конструкции *letrec-syntax* вполне себе разрешимая задача. По аналогии с *letrec* она позволяет создавать локальные трансформаторы синтаксиса, которые могут ссылаться на себя

Сначала потренируемся:

```
(letrec-syntax ((exec-rule ; exec-rule -- это форма, которая внутри себя может использовать
                          ; себя же рекурсивно
  (syntax-rules (or and not)
    ; описываем правила с учетом того, что or, and и not -- это лексические константы
    ((exec-rule (and x ...)) `(* ,(exec-rule x) ...)) ; and переводим в умножение
    ((exec-rule (or x ...)) `(max ,(exec-rule x) ...)) ; or -- в max
    ((exec-rule (not x)) `(- 1 ,(exec-rule x))) ; not
    ((exec-rule x) 'x))) ; всё остальное пока оставляем как есть
  ; пробуем применить
  (exec-rule (and (or 0.1 0.2) (not 0.9) (and 0.5 0.3))))
=>
(* (max 0.1 0.2) (- 1 0.9) (* 0.6 0.3))
```

¹Здесь используется конструкция "квазигитирования" `(... ,(...) ...). Она, в отличие от обычного цитирования - *quote*, которое формируется при помощи '(...), позволяет вставлять в цитату значения переменных или даже результат вычисления функций.

Здесь мы воспользовались *квазицитированием* с целью создать внутреннюю структуру вычислений исходя из правила, а не произвести сами вычисления.

Без квазицитирования наш синтаксический трансформатор выглядит так:

```
(letrec-syntax ((exec-rule
  (syntax-rules (or and not)
    ((exec-rule (and x ...)) (* (exec-rule x) ...))
    ((exec-rule (or x ...)) (max (exec-rule x) ...))
    ((exec-rule (not x)) (- 1 (exec-rule x)))
    ((exec-rule x) x))))
  ; пробуем применить
  (exec-rule (and (or 0.1 0.2) (not 0.9) (and 0.5 0.3))))
=>
0.003
```

И, соответственно, он будет заниматься реальными вычислениями, а не построением S-выражений. Теперь у нас готова заготовка для самой сложной части – применения логических правил. Не забудем, что последнее правило трансформации `((exec-rule x) x)` нужно будет немного преобразовать, чтобы оно работало с термами нечёткой логики.

2.2.4. Дефазификация

Вычисления по правилам должны привести к вектору принадлежности результирующей переменной. Т.е. будем считать, что каждое правило после своего применения дало вес тому или иному нечёткому множеству n . Далее, из вектора весов V_n и функций принадлежности μ_n нам необходимо составить весовую функцию $w(y)$, затем найти её "центр масс":

```
; var -- alist с описанием переменной (её функции принадлежности)
; v -- alist с вектором весов
(define (weight var v)
  (lambda (y) ; вернуть функцию от y
    (apply max ; которая рассчитывает максимум на
      (map ; множестве
        (lambda (r) (* ; произведений
          (cdr r) ; значений вектора веса для множества r
          ((assq-ref var (car r)) y))) ; на значение функции принадлежности
        ; этому множеству
      v)))) ; для каждого элемента вектора веса
```

Весовую функцию надо ещё проинтегрировать и найти её "центр масс". Обычно задача интегрирования сводится к суммированию по фиксированному набору точек. Выбирается разумный диапазон и шаг. Ошибки такого интегрирования ничуть не больше ошибок фаззификации. Поэтому можно особо не усердствовать:

```
; from, to -- диапазон интегрирования
; divisions -- количество делений диапазона
; var -- alist с описанием переменной
; v -- alist с вектором весов
(let ((w (weight var v))) ; запомним весовую функцию
  (let loop ; начнем "цикл" по разбиению.
    ((wv 0) ; начальное значение верхнего интеграла
     (tw 0) ; начальное значение нижнего интеграла
     (t from) ; начинаем с точки from
     (dt (/ (- to from) divisions))) ; шаг интегрирования
    (if (<= t to)
      (loop (+ (* t (w t)) wv) ; верхний интеграл
```

```

      (+ (w t) tw) ; нижний интеграл
      (+ t dt) ; инкремент шага
      dt)
; когда дошли до конца, возвращаем отношение
(/ wv tw))))

```

Теперь объединяем все это вместе под одним *define-syntax*

```
(use-module (srfi srfi-1)) ; этот модуль нужен для assq-ref
```

```
(define-syntax fuzzy ; определяем синтаксическую форму fuzzy
  (syntax-rules (if) ; if нам потребуется как константа
    ((_ resvar (from to divisions) ; на всякий случай передадим области и
      ; шаг интегрирования результата
      ((varname ; имя нечёткой переменной
        (range pf params ...) ; название нечёткого множества
        ; имя функции принадлежности и
        ; ее параметры
        ...) ; продолжаем описывать множества сходным образом
      ...) ; продолжаем описывать переменные
      ; теперь описываем правила
      (vr if rule) ; правила в форме название множества if правило
      ...) ; это всё синтаксический шаблон
    ; теперь его преобразование
    (lambda (varmap) -- вернуть функцию от alist со значениями переменных
      (let ((varname `((range . ,(pf params ...)) ; для каждой переменной
        ...) ; строим alist из пар
        ; range - функция принадлежности
        (let ((result '())) ; обозначим результат в виде alist, пока пустого
          (letrec-syntax ((exec-rule ; вставляем заготовку рекурсивной обработки
            ; правил
            (syntax-rules ::: (or and not is) ; используем внутри эллипсис :::
              ; чтобы не путать с ...
              ; и определяем лексические конст
              ((exec-rule (and p :::)) (* (exec-rule p) :::))
              ((exec-rule (or p :::)) (max (exec-rule p) :::))
              ((exec-rule (not p)) (- 1 (exec-rule p)))
              ((exec-rule (qv is rng)) ; если встретили терм принадлежности q
                ; преобразовываем в функцию вычисления принадлежности
                ((assq-ref qv 'rng) (assq-ref varmap 'qv))))))
            ; в рамках этого синтаксиса строим alist в весах
            (begin ; для каждого правила
              (set! result (acons 'vr (exec-rule rule) result)) ...))
          ; теперь можно интегрировать
          (let ((w (weight resvar result)))
            (let loop ((wv 0) (tw 0) (t from) (dt (/ (- to from) divisions)))
              (if (< v to)
                (loop (+ (* t (w t)) wv) (+ (w t) tw) (+ t dt) dt)
                (/ wv tw)))))))))) ; уфф!

```

Теперь можно и попробовать

```
(define speed-func (fuzzy speed (0 500 100)
  ((temp (low trap -200 -200 -20 5)
    (med trap -20 5 5 15)

```

```

        (high trap 5 15 60 60))
(humid (low trap 0 0 10 20)
        (med trap 10 20 40 50)
        (high trap 40 50 100 100))
(pres (low trap 0 0 700 750)
       (med trap 700 750 760 780)
       (high trap 760 780 1000 1000))
(speed (low trap 0 0 100 200)
        (med trap 100 200 300 400)
        (high trap 300 400 500 500)))
(high if (and (temp is high) (or (humid is high) (pres is low))))
(med if (and (temp is med) (or (humid is high) (not (pres is med)))))
(low if (or (temp is low) (humid is low) (pres is high))))

(speed-func '((temp . 10) (humid . 30) (pres . 770)))
=>
144.47

```

Задания

1. Напишите цикл по одной или двум переменным и выведите график среза нечёткой функции (можно использовать утилиту gnuplot).
2. Преобразуйте синтаксис правил так, чтобы логические связки записывались в инфиксной форме:

```
((temp is high) and ((pres is low) or (humid is low)))
```

3. Преобразуйте синтаксис, чтобы можно было описывать нечёткие функции с именованными параметрами. Например:

```
(fuzzy (temp humid pres) speed ((temp (...)) (humid ...)))
```

Так, чтобы её можно было использовать простым образом:

```
(speed-func 10 30 770)
```